```
else
  puts "no integers"
end
```

produces:

```
all integers, the middle one is 2
```

There are a couple of shortcuts. If you don't need to pattern match the part of the pattern with the local variable, you can just use the variable name without the hashrocket:

```
case [1, 2, 3]
in [Integer, middle, Integer]
  puts "The middle one is #{middle}"
in [*, Integer, *]
  puts "contains an integer"
else
  puts "no integers"
end
```

produces:

```
The middle one is 2
```

In a hash, you can bind a variable to the same name as the key of the hash by just including the key:

```
case {first_name: "Ron", last_name: "Lithgow"}
in {first_name:}
  puts "this person's name is #{first_name}"
else
  puts "this data does not have a first name"
end
```

produces:

```
this person's name is Ron
```

The splatted parts of an array or hash can be bound to a local variable by putting the variable name after the splat, as in [Integer, Integer, *rest].

There is one limitation, if you use the | character to match multiple patterns, you can not bind variables inside any of the multiple patterns, this is a limitation of the pattern matching parser.

If you want to reuse an existing variable, local, global, or instance, as part of a pattern, or if you want to set a variable at the beginning of a pattern and use it later in the pattern, a ^ caret in front of the variable name means "use the existing value for this variable".

```
count = 2
```

```
case [1, 2, 3]
in [Integer, ^count, last]
  puts "The middle one 2, the last one is #{last}"
in [*, Integer, *]
  puts "contains an integer"
else
  puts "no integers"
end
```